

Employee Compensation Service - Interview Quick Reference

1. Tech Stack & Workflow (The 30-Second Summary)

What Technologies We Used:

- **Python:** Programming language for all backend logic.
- **Azure Functions (Python V2):** Serverless API framework.
- **Azure SQL Database (Serverless):** Cloud database for live data storage.
- **SQLite:** File-based database for quick local testing.
- **Git & GitHub:** Version control to track and host the code.

How We Used Them:

1. **Local Testing:** Wrote the code locally in Python and tested it using a local SQLite file (*employee_comp.db*).
2. **Going Live:** Deployed the code to the Azure cloud using a single command: `npx func azure functionapp publish`.
3. **Connecting to SQL:** In the cloud, the code reads the connection credentials from Environment Settings and uses the **pyodbc** driver to connect directly to the live **Azure SQL Database**.
4. **Automatic Alerts:** Database updates trigger an HTTP POST webhook request to **Azure Logic Apps** to email notifications to the HR team.

2. Layer-by-Layer Request Flow

Layer	Component	Description
Layer 1: Client	Postman / Browser	Sends HTTP requests (e.g. GET /api/employees)
Layer 2: Routing	function_app.py	Maps routes dynamically using Blueprint route decorators
Layer 3: Logic	employees.py / reporting.py	Enforces default 5% bonus logic and aggregates reports
Layer 4: Data Access	database.py	Auto-detects environment; executes queries via pyodbc/sqlite3
Layer 5: Database	Azure SQL / SQLite file	Performs DB executions and returns database tuples

3. Project File Catalog

File Path	Detailed Operation & Purpose
function_app.py	Initializes the Function App container and registers dynamic blueprints route handlers.
host.json	Controls general configuration settings and telemetry logging parameters for the Azure host.
local.settings.json	Stores local environment database credentials for offline testing (hidden from Git).
requirements.txt	Defines external pip modules like pyodbc and azure-functions for Oryx compilation.
schema.sql	Contains DDL tables schema creation code and initial pre-seeded employee data records.
.gitignore	Blocks sensitive local configurations and SQLite databases from leaking to public repos.
.funcignore	Excludes large node packages, virtual environments, and local databases from deployment zip.

4. In-Depth File-by-File Operations Guide

src/database.py

Implements the DatabaseManager class which abstracts connectivity. If the connection string contains Driver=, it initiates a connection using pyodbc for the live Azure SQL Database. Otherwise, it defaults to a local SQLite database file (sqlite3.connect). It also includes a formatting helper to map table tuples to dictionary rows.

src/functions/employees.py

Handles the CRUD operations. Includes conditional check logic: if an employee is created or updated with no bonus, it automatically computes the 5% bonus, writes it, and invokes an HTTP client webhook trigger to Azure Logic Apps.

src/functions/reporting.py

Defines the aggregate calculation endpoints. Implements COALESCE to convert NULLs to 0.0 in reports, performs mathematical casts to prevent integer truncation errors, and writes grouping queries to rank employee comp details.

src/seed.py

An offline helper script that initializes and seeds the local SQLite database file (*employee_comp.db*) using schema definitions.